

Phase 1— Create the VPC

Create VPC

You should see options such as:

- VPC only
- VPC and more

For learning, choose:

VPC only

The screenshot shows the AWS Management Console interface for a VPC named 'vpc-0440b8a09a9ddf7c6 / three-tier-vpc'. The 'Details' tab is selected, displaying various configuration options. The VPC ID is 'vpc-0440b8a09a9ddf7c6', and its state is 'Available'. The 'Block Public Access' is set to 'Off'. The 'DNS hostnames' are 'Disabled'. The 'Main network ACL' is 'acl-0a985879dec9cbf65'. The 'IPv6 CIDR (Network border group)' is empty. The 'Encryption control ID' is empty. The 'Tenancy' is 'default'. The 'Default VPC' is 'No'. The 'Network Address Usage metrics' are 'Disabled'. The 'Encryption control mode' is empty. The 'DHCP option set' is 'dopt-07991c3e6d7b02628'. The 'IPv4 CIDR' is '10.0.0.0/16'. The 'Route 53 Resolver DNS Firewall rule groups' are empty. The 'Main route table' is 'rtb-0fdbee462efa7535'. The 'IPv6 pool' is empty. The 'Owner ID' is '181188393114'. Below the details, there is a 'Resource map' section showing the VPC, Subnets (0), Route tables (1), and Network Connections (0).

Phase 4 — Create Internet Gateway

three-tier-igw

The screenshot shows the AWS Management Console interface for an Internet Gateway named 'three-tier-igw'. A green notification bar at the top states: 'The following internet gateway was created: igw-0dd5e9902dcc3c905 - three-tier-igw. You can now attach to a VPC to enable the VPC to communicate with the internet.' Below this, the 'Attach to VPC (igw-0dd5e9902dcc3c905)' dialog is open. The dialog prompts the user to 'Attach an internet gateway to a VPC to enable the VPC to communicate with the internet. Specify the VPC to attach below.' It lists 'Available VPCs' and shows a search bar with 'vpc-0440b8a09a9ddf7c6' entered. The search results show 'vpc-0440b8a09a9ddf7c6 - three-tier-vpc'. The 'Attach to VPC' button is highlighted in orange.

Phase 5 — Create 6 subnets

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

public-subnet-az1

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.1.0/24 256 IPs

Tags - optional

Key Value - optional

Name public-subnet-az1 Remove

Add new tag

You can add 49 more tags.

Subnet settings

Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

private-web-subnet-az1

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Asia Pacific (Mumbai) / ap-south-1a

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16

IPv4 subnet CIDR block

10.0.11.0/24 256 IPs

Tags - optional

Key Value - optional

Name private-web-subnet-az1 Remove

Add new tag

You can add 49 more tags.

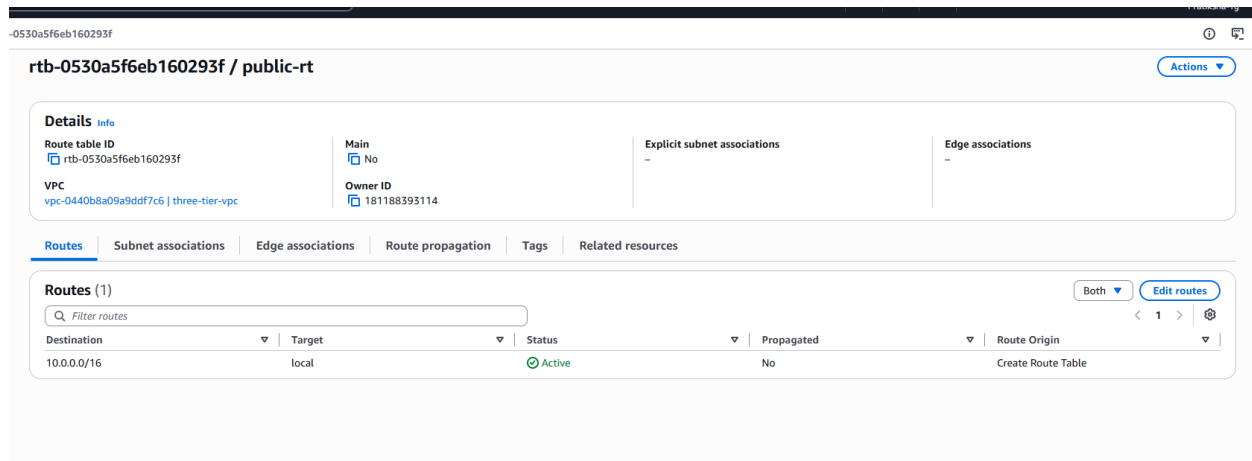
You have successfully created 1 subnet: subnet-0e2cc1a3b4a815bed

Subnets (9) [Info](#)

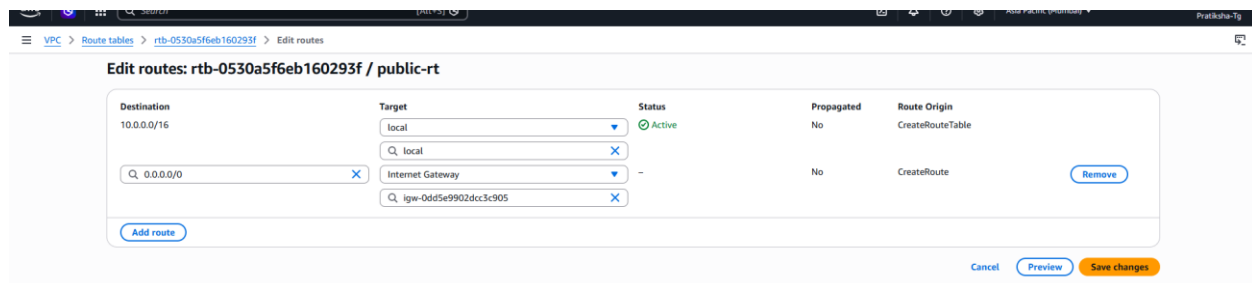
Find subnets by attribute or tag

	Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6
☐	public-subnet-az1	subnet-0b5e8dc32d1579f31	Available	vpc-0440b8a09a9dd7c6 three...	Off	10.0.1.0/24	-	-
☐	private-web-subnet-az1	subnet-0d9e1a86aec7f144d	Available	vpc-0440b8a09a9dd7c6 three...	Off	10.0.11.0/24	-	-
☐	private-db-subnet-az1	subnet-025ff4ceea591a483	Available	vpc-0440b8a09a9dd7c6 three...	Off	10.0.21.0/24	-	-
☐	public-subnet-az2	subnet-03b57425bebe41e07	Available	vpc-0440b8a09a9dd7c6 three...	Off	10.0.2.0/24	-	-
☐	private-web-subnet-az2	subnet-09a29e782be370fc1	Available	vpc-0440b8a09a9dd7c6 three...	Off	10.0.12.0/24	-	-
☐	private-db-subnet-az2	subnet-0e2cc1a3b4a815bed	Available	vpc-0440b8a09a9dd7c6 three...	Off	10.0.22.0/24	-	-

Now let's create the Public Route Table



Add the Internet Gateway route



Associate the two public subnets:

While public-rt is selected, go to:

Subnet associations

→ **Edit subnet associations**

Select **only**:

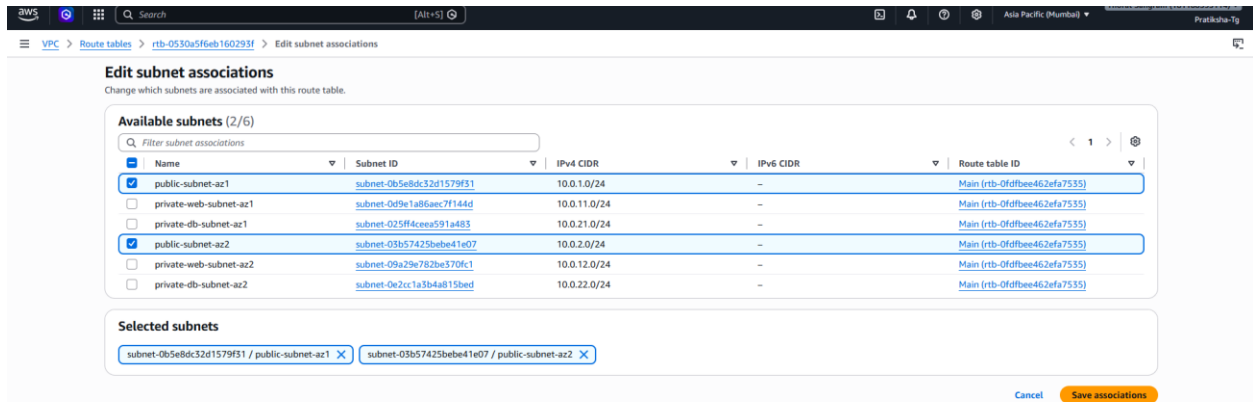
- ☒ public-subnet-az1
- ☒ public-subnet-az2

Do **not** select:

- ☐ private-web-subnet-az1
- ☐ private-web-subnet-az2
- ☐ private-db-subnet-az1
- ☐ private-db-subnet-az2

Click:

Save associations



STEP 1 — Create NAT Gateway

Go to:

VPC → NAT gateways

You can see **NAT gateways** in the left menu of your screenshot.

Click it.

Then click:

Create NAT gateway

STEP 2 — Enter NAT Gateway details

You'll see something similar to:

Name

Enter:

three-tier-nat-az1

Subnet

Select:

public-subnet-az1

Connectivity type

Select:

Public

Elastic IP allocation

Click:

Allocate Elastic IP

AWS will allocate an Elastic IP.

Then click:

Create NAT gateway

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Tharsh Sangam (181168393114)

Pratiksha-Tp

VPC > NAT gateways > Create NAT gateway

Elastic IP address 15.252.172.52 (eipalloc-049494b6d3dfa329) allocated.

Create NAT gateway

A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.

NAT gateway settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

The name can be up to 256 characters long.

Availability mode Info
Choose whether to deploy across all zones in the region or restrict to a single availability zone.

☒ **Regional - new**
Scales automatically across all regional AZs, simplifying management for multi AZ deployments.

☐ **Zonal**
Provides granular control within a specific availability zone, adhering to subnet level settings.

VPC
Select a VPC in which to create the regional NAT gateway.

vpc-0440b8a09a9dd7c6 (three-tier-vpc)

Connectivity type
Select a connectivity type for the NAT gateway.

☒ **Public**

☐ Private

Method of Elastic IP (EIP) allocation Info
Choose how IP addresses are associated with NAT gateways.

☐ **Automatic**
AWS automatically manages EIPs and AZ coverage for NAT gateways. This ensures easy scaling—adding AZs automatically allocates EIPs, simplifying management.

☒ **Manual**
Manually assigns specific IP addresses for compliance or whitelisting. Note: Requires manual scaling to new AZs as workloads expand.

CloudShellAgent Toolkit for AWSFeedbackConsole mobile app

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

ENG IN17:4723-09-2026

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Tharsh Sangam (181168393114)

Pratiksha-Tp

VPC > NAT gateways > Create NAT gateway

Elastic IP address 15.252.172.52 (eipalloc-049494b6d3dfa329) allocated.

☐ Automatic
AWS automatically manages EIPs and AZ coverage for NAT gateways. This ensures easy scaling—adding AZs automatically allocates EIPs, simplifying management.

☒ Manual
Manually assigns specific IP addresses for compliance or whitelisting. Note: Requires manual scaling to new AZs as workloads expand.

Elastic IP (EIP) association
Associate EIPs to Availability Zones to ensure each availability zone has its own internal exit point, enhancing fault tolerance and maintaining network connectivity.

Availability zones
aps1-az1 (ap-south-1a)
aps1-az3 (ap-south-1b)
aps1-az2 (ap-south-1c)

Elastic IPs

Select one or more EIPs
13.206.104.78 X

Select one or more EIPs
15.252.172.52 X

Select one or more EIPs

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key
 X

Value - optional
 X

Remove

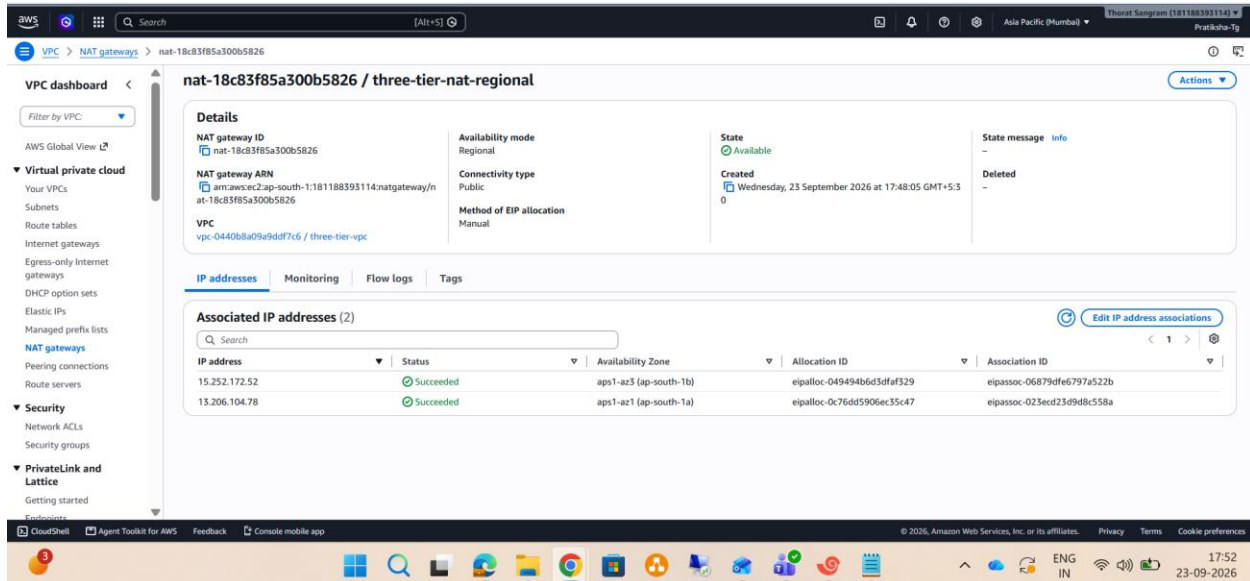
Add new tag
You can add 49 more tags.

CancelCreate NAT gateway

CloudShellAgent Toolkit for AWSFeedbackConsole mobile app

© 2026, Amazon Web Services, Inc. or its affiliates. PrivacyTermsCookie preferences

ENG IN17:4723-09-2026



Private Web Route Table — AZ-1

Purpose

Your Ubuntu EC2 will eventually live here:

private-web-subnet-az1

Because this subnet is private, the EC2 won't have a public IP.

But it may need outbound internet access for things like:

```
apt update
apt install nginx
wget ...
curl ...
```

So we'll create a route:

Private EC2

|

v

Private Web Route Table

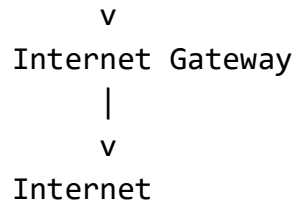
|

| 0.0.0.0/0

v

Regional NAT Gateway

|



This is **outbound internet access**, not direct inbound access.

Step 1 — Go to Route Tables

In the left menu click:

Route tables

Then click:

Create route table

Step 2 — Create the route table

Enter:

Name

private-web-rt-az1

VPC

Select:

three-tier-vpc

Then click:

Create route table

Create route table [Info](#)

A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.

Route table settings

Name - optional
Create a tag with a key of 'Name' and a value that you specify.

private-web-rt-az1

VPC
The VPC to use for this route table.

vpc-0440b8a09a9dd7c6 (three-tier-vpc)

Tags
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key **Value - optional**

Q Name X Q private-web-rt-az1 X Remove

[Add new tag](#)

You can add 49 more tags.

[Cancel](#) [Create route table](#)

Step 3 — Add the NAT route

Select the newly created:

private-web-rt-az1

Go to:

Routes → Edit routes

You should see:

10.0.0.0/16 → local

 **Why is this route already there?**

AWS automatically adds the VPC's local route.

It means:

Traffic destined for resources inside 10.0.0.0/16 stays inside the VPC.

For example:

10.0.11.10 → 10.0.21.10

doesn't need the NAT Gateway.

It uses:

10.0.0.0/16 → local

Now add the internet route

Click:

Add route

Destination:

0.0.0.0/0

Target:

NAT Gateway

Select:

three-tier-nat-regional

or:

nat-18c83f85a300b5826

Your final routes should be:

Destination	Target

10.0.0.0/16	local
0.0.0.0/0	NAT Gateway

Click:

Save changes

Destination: 10.0.0.0/16

Target: local

Status: Active

Propagated: No

Route Origin: CreateRouteTable

Destination: 0.0.0.0/0

Target: NAT Gateway

Status: -

Propagated: No

Route Origin: CreateRoute

Buttons: Add route, Remove, Cancel, Preview, Save changes

Step 4 — Associate AZ-1 Web Subnet

Now go to:

Subnet associations

Click:

Edit subnet associations

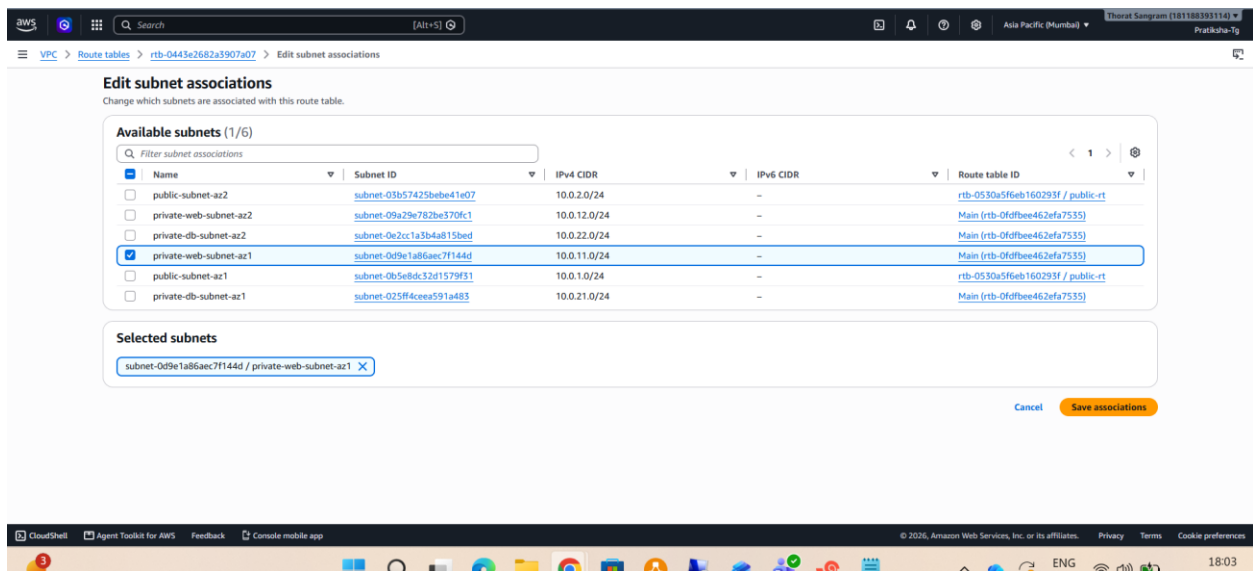
Select:

☒ private-web-subnet-az1

Don't select the other five.

Click:

Save associations



Create Private Web Route Table for AZ-2

Why are we creating another one?

You have:

AZ-1

Private Web 1

10.0.11.0/24

AZ-2

Private Web 2

10.0.12.0/24

We want both web subnets to have outbound internet through NAT.

For clarity and independent control, we'll create:

private-web-rt-az1 → private-web-subnet-az1

private-web-rt-az2 → private-web-subnet-az2

1. Go to Route Tables

In the left menu click:

Route tables

Then click:

Create route table

2. Enter these values

Name

private-web-rt-az2

VPC

Select:

three-tier-vpc

Then click:

Create route table

The screenshot shows the AWS Management Console interface for creating a new route table. The breadcrumb navigation at the top indicates the path: VPC > Route tables > Create route table. The main heading is 'Create route table' with an 'Info' link. Below this is a descriptive sentence: 'A route table specifies how packets are forwarded between the subnets within your VPC, the internet, and your VPN connection.'

The 'Route table settings' section contains two fields: 'Name - optional' with the value 'private-web-rt-az2' and 'VPC' with a dropdown menu showing 'vpc-0440b8a09a9dd7c6 (three-tier-vpc)'. Below this is the 'Tags' section, which includes a description of tags and a table with one tag: 'Name' as the key and 'private-web-rt-az2' as the value. There are 'Add new tag' and 'Remove' buttons. At the bottom right, there are 'Cancel' and 'Create route table' buttons.

3. Add the routes

Select:

private-web-rt-az2

Go to:

Routes → Edit routes

You should already have:

10.0.0.0/16 → local

Click **Add route**.

Enter:

Destination:

0.0.0.0/0

Target:

NAT Gateway

Select:

three-tier-nat-regional

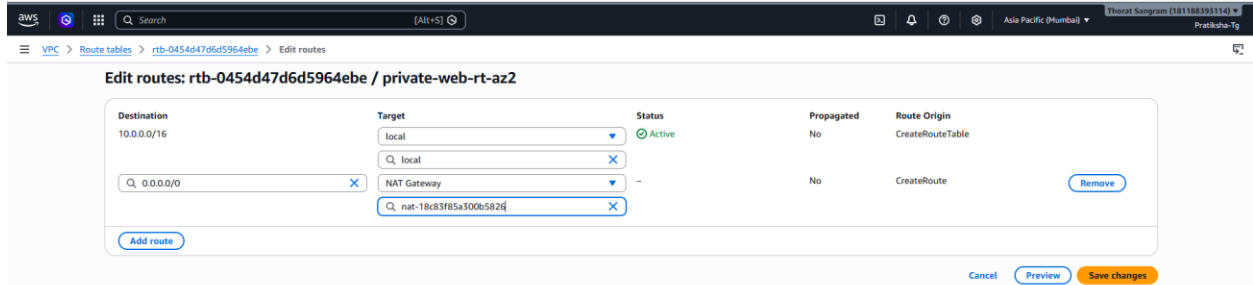
Your final routes should be:

Destination	Target

10.0.0.0/16	local
0.0.0.0/0	nat-18c83f85a300b5826

Click:

Save changes



4. Associate the AZ-2 private web subnet

Now go to:

Subnet associations

Click:

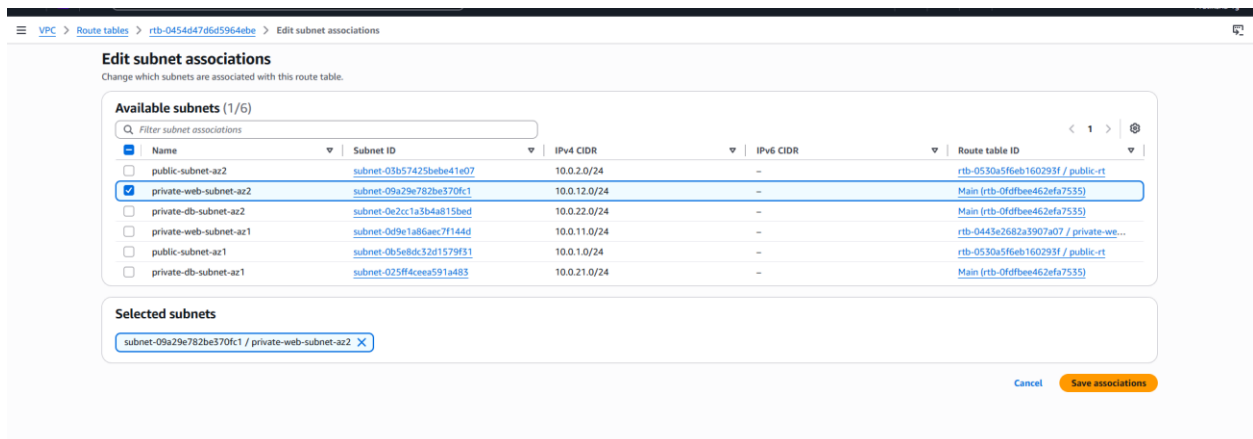
Edit subnet associations

Select ONLY:

☒ private-web-subnet-az2

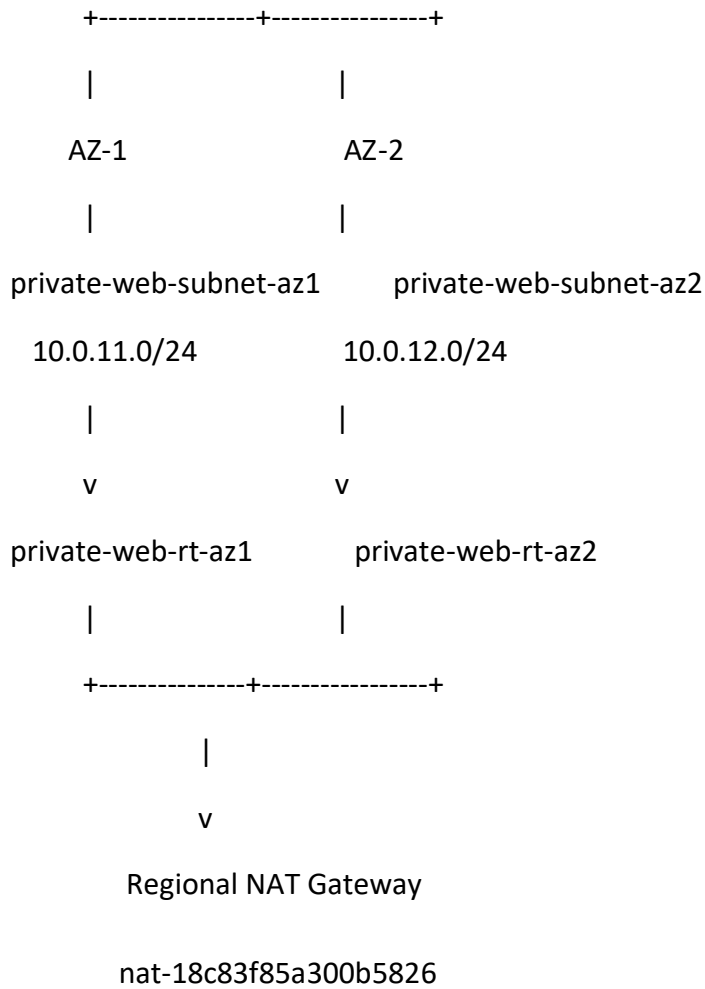
Then:

Save associations



three-tier-vpc

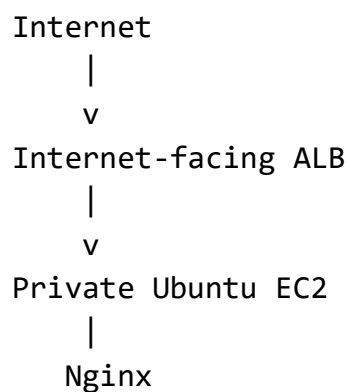
|



Next step: Create Security Groups

Now we're moving from **routing** to **security**.

Our target is:



|
Application

We need to control **who can talk to whom**.

We'll create **2 Security Groups** for this POC:

1. alb-sg

Controls traffic coming to the Load Balancer.

2. web-sg

Controls traffic coming to the private EC2 servers.

Step 1 — Create alb-sg

Go to:

AWS Console → VPC → Security groups

Click:

Create security group

Security group name

alb-sg

Description

Security group for internet-facing Application Load Balancer

VPC

Select:

three-tier-vpc

Inbound rules

Add:

Type	Port	Source	Why
HTTP	80	0.0.0.0/0	Allow users from internet to access ALB

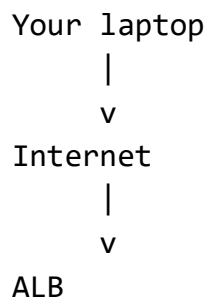
So it should look like:

HTTP
TCP
80
0.0.0.0/0

Why 0.0.0.0/0?

Because your ALB is the **public entry point**.

Users could come from anywhere:



Therefore HTTP 80 needs to accept internet traffic.

For this POC, we'll use HTTP first. We can add HTTPS later.

Outbound rules

Leave the default:

All traffic
0.0.0.0/0

This allows the ALB to communicate with the backend targets.

Click:

Create security group

The screenshot displays the AWS Management Console interface for creating a new security group. The breadcrumb navigation shows 'VPC > Security Groups > Create security group'. The selected VPC is 'vpc-0440b8a09a9d0d7c6 (three-tier-vpc)'. The 'Inbound rules' section has a table with one rule: Type 'HTTP', Protocol 'TCP', Port range '80', Source 'Anywhere...', and a destination of '0.0.0.0/0'. A warning message below states: 'Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.' The 'Outbound rules' section also has a table with one rule: Type 'All traffic', Protocol 'All', Port range 'All', Destination 'Custom', and a destination of '0.0.0.0/0'. Another warning message states: 'Rules with destination of 0.0.0.0/0 or ::/0 allow your instances to send traffic to any IPv4 or IPv6 address. We recommend setting security group rules to be more restrictive and to only allow traffic to specific known IP addresses.' The 'Tags - optional' section is empty. The footer shows navigation links, copyright information, and system details like 'ENG IN' and '18:30 23-09-2026'.

Step 2 — Create web-sg

Now create another Security Group.

Name

web-sg

Description

Security group for private web EC2 instances

VPC

three-tier-vpc

Inbound rule

Add:

Type: HTTP

Port: 80

Source: Custom

For the source, select:

alb-sg

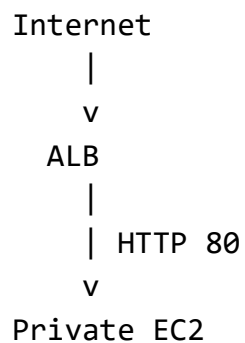
 **Very important**

Do **NOT** use:

0.0.0.0/0

for the EC2 HTTP rule.

We want:



Not:



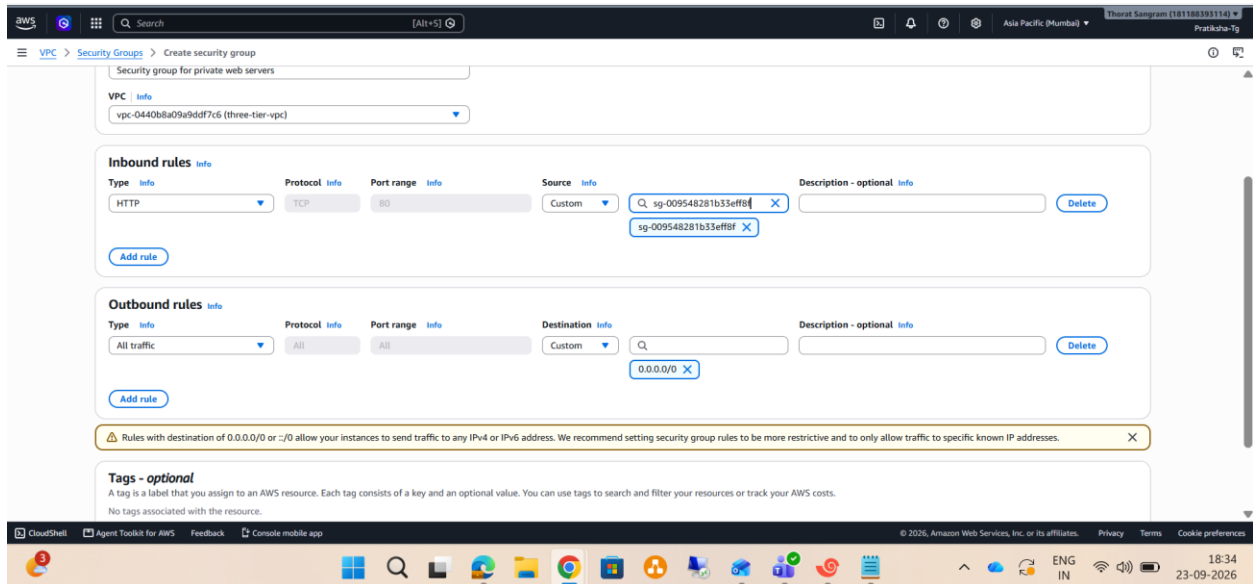
The EC2 should only accept HTTP traffic from the ALB.

Outbound

Leave:

All traffic

0.0.0.0/0



This allows the private EC2 to reach the internet through your NAT Gateway when required.

For example:

```
sudo apt update
sudo apt install nginx
```

Launch the EC2 Server:

Step 1 – Create IAM Role for the EC2

Because the EC2 is private and has no public IP, we need a way to access it.

We'll use **Systems Manager Session Manager**.

Go to:

AWS Console → IAM → Roles → Create role

Trusted entity

Select:

Trusted entity type:

AWS service

For service/use case:

Service:

EC2

Click **Next**.

Step 2 – Add permission

Search:

AmazonSSMManagedInstanceCore

Select the AWS-managed policy:

AmazonSSMManagedInstanceCore

Click **Next**.

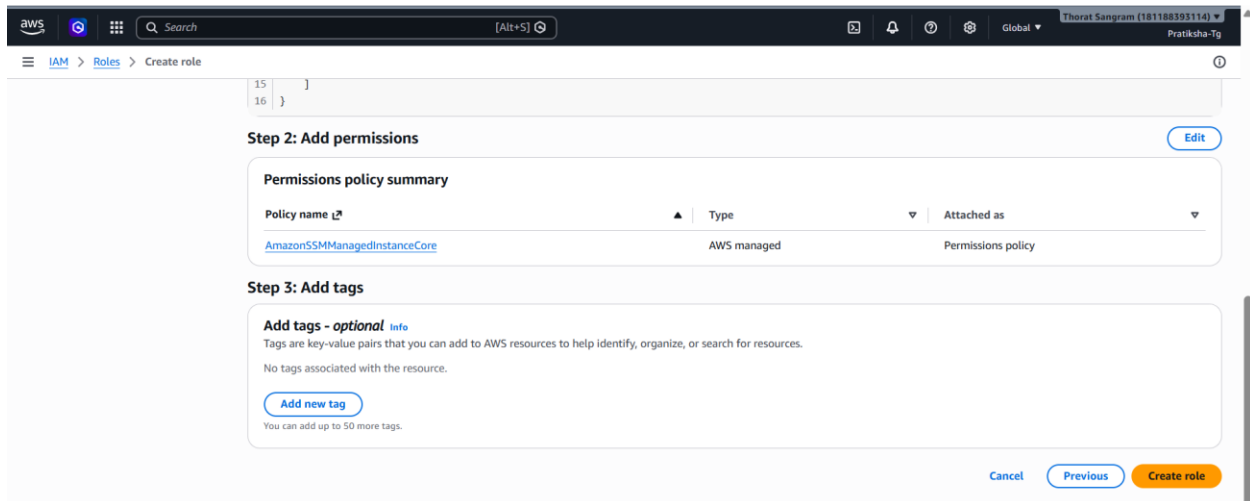
Role name

Enter:

EC2-SSM-Role

Click:

Create role



Step 3 – Attach the role to your existing EC2

Go to:

EC2 → Instances

Select:

three-tier-web-az1

Then:

Actions → Security → Modify IAM role

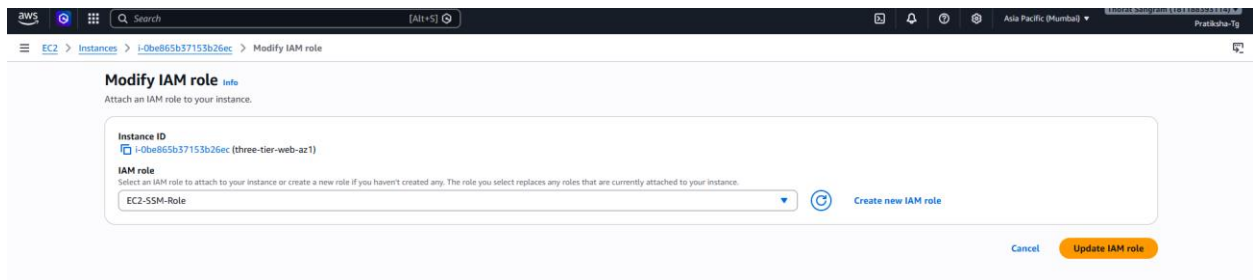
You should see the IAM instance profile option.

Select:

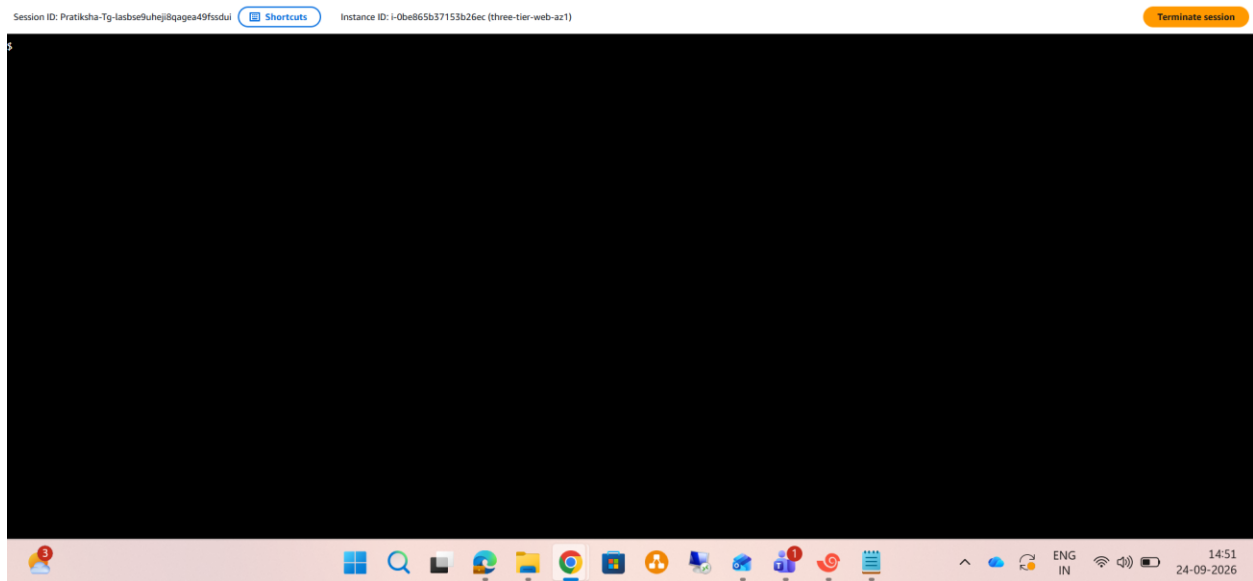
EC2-SSM-Role

Click:

Update IAM role



Connect using session manager:



Run these **three commands**:

```
sudo apt update
sudo apt install nginx -y
sudo systemctl status nginx --no-pager
```

Step 1 — Create index.html

Run this in your Session Manager terminal:

```
sudo nano /var/www/html/index.html
```

Then paste:

```
<!DOCTYPE html>
<html>
<head>
```



```
<title>Three Tier Architecture</title>
</head>
<body>

  <h1>Welcome to This Task!</h1>

  <h2>AWS Three-Tier Architecture POC</h2>

  <p>Web Server: AZ1</p>
  <p>Server IP: 10.0.11.14</p>

  <p>Nginx is running successfully.</p>

</body>
</html>
```

Step 2 — Save the file

In nano:

```
Ctrl + O
Enter
Ctrl + X
```

Step 3 — Test the webpage

Run:

```
curl http://localhost
```

Next Step – Create the Target Group

1. Go to Target Groups

AWS Console:

EC2 → Load Balancing → Target Groups

Click:

Create target group

2. Choose target type

Select:

Instances

This means we will register our EC2 instance directly.

Click **Next**.

3. Basic configuration

Set:

Target group name:

three-tier-web-tg

Protocol:

HTTP

Port:

80

VPC:

three-tier-vpc

4. Health check

Keep:

Health check protocol: HTTP

Health check path: /

Why /?

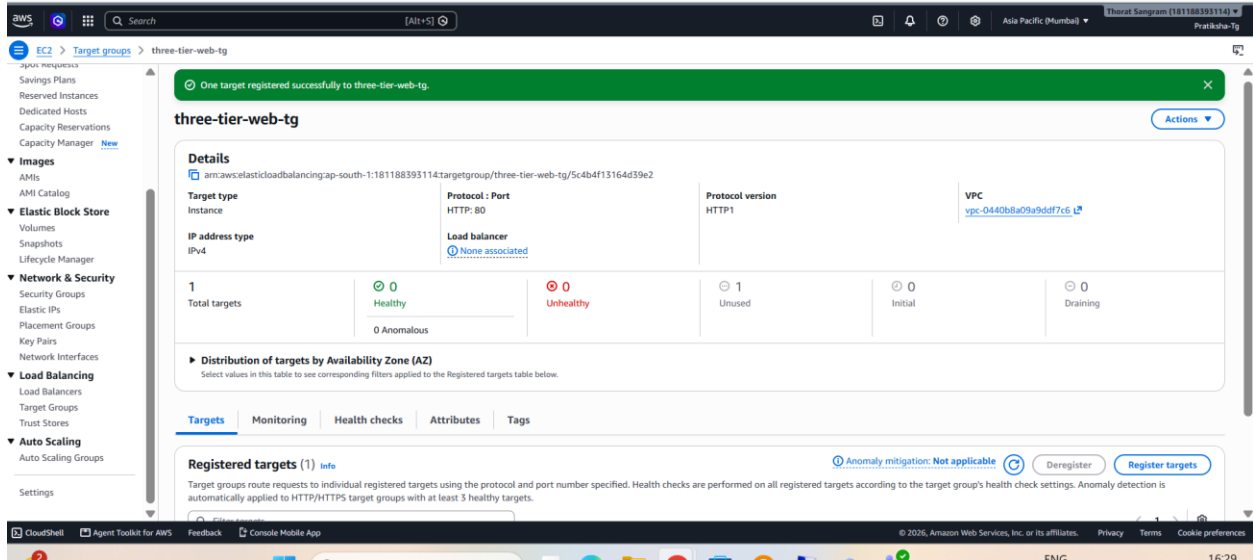
Because our Nginx website is available at:

<http://10.0.11.14/>

The ALB will request / and expect a successful response.

5. Register the EC2

Click **Next**.



Step 1 — Go to Load Balancers

In AWS:

EC2 → Load Balancing → Load Balancers

Click:

Create Load Balancer

You should see several options.

Select:

Application Load Balancer

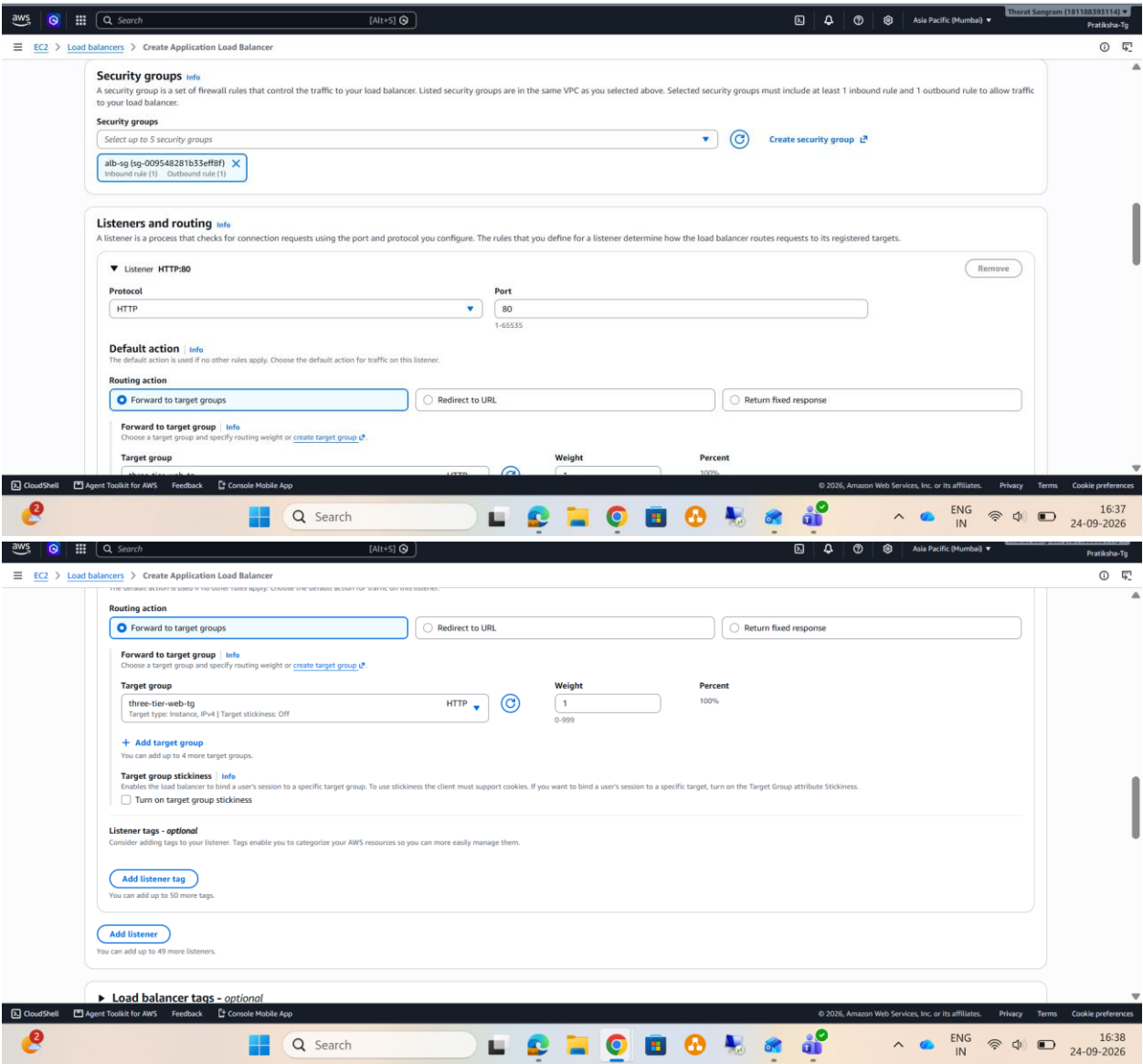
Why?

Because we're serving an HTTP/HTTPS web application.

Then click:

Create

Setting	Value
Name	three-tier-alb
Scheme	Internet-facing
IP address type	IPv4



Configuration	Expected
Load balancer type	Application Load Balancer
Name	three-tier-alb
Scheme	Internet-facing

IP type	IPv4
VPC	three-tier-vpc
Subnet 1	public-subnet-az1
Subnet 2	public-subnet-az2
Security group	alb-sg
Listener	HTTP :80
Forward to	three-tier-web-tg

aws [Search] [Alt+S] Ask Amazon Q Asia Pacific (Mumbai) Threat Sangam (181163323154) Pratikho-Tp

EC2 > Target groups > three-tier-web-tg

Details

amazonaws:elasticloadbalancing:ap-south-1:181188393114:targetgroup/three-tier-web-tg/5c4b4f13164d39e2

Target type: Instance

Protocol : Port: HTTP: 80

Protocol version: HTTP1

VPC: vpc-0440b8a09a9dd7c6

IP address type: IPv4

Load balancer: three-tier-alb

1 Total targets

1 Healthy

0 Unhealthy

0 Anomalous

0 Unused

0 Initial

0 Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets Monitoring Health checks Attributes Tags

Registered targets (1) info

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Anomaly mitigation: Not applicable

Deregister Register targets

Filter targets

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details
☐	i-0be865b37153b25ec	three-tier-web-az1	80	ap-south-1a (a...	Healthy	-	No override	No override is curren...

Load balancers (1/1) What's new?

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security groups	DNS name
☒	three-tier-alb	Active	application	Internet-facing	IPv4	vpc-0440b8a09a9dd7c6	2 Availability Zones	sg-009548281b33eff8f	three-tier-alb-9

Load balancer: three-tier-alb

Listeners and rules (1) info

A listener checks for connection requests on its configured protocol and port. Incoming requests are evaluated against rules in priority order; the first matching rule determines routing. If no rule matches, the default action is applied.

Filter listeners

☐	Protocol:Port	Default action	Rules	ARN	Security policy	Default SSL/TLS certificate	mTLS	Trust sto
☐	HTTP:80	Forward to target group three-tier-web-tg (1 (100%)) Target group stickiness: Off	1 rule	ARN	Not applicable	Not applicable	Not applicable	Not appl

Not secure three-tier-alb-920519494.ap-south-1.elb.amazonaws.com

cloud-hindavitech... ChatGPT cloud-hindavitech... New tab Add service - AWS P... Login || Sangam CRM Jira Site24x7

Welcome to This Task!

AWS Three-Tier Architecture POC

Web Server: AZ1

Server IP: 10.0.11.14

Nginx is running successfully.

